

THEORY -

1. What is data wrangling? Explain the difference between data wrangling and data preprocessing.**

Data wrangling is the process of transforming and organizing raw data into a structured format suitable for analysis. It includes tasks like cleaning, filtering, merging, and reshaping data.

Difference:

- **Data Wrangling:** Focuses on transforming and structuring data (e.g., merging datasets, reshaping tables).
- **Data Preprocessing:** Focuses on preparing data for machine learning (e.g., normalization, encoding, scaling).

2. What is the importance of exploratory data analysis (EDA) in Pandas?**

Exploratory Data Analysis (EDA) is used to understand the dataset before applying any analysis or model.

Importance:

- Helps in understanding data patterns and trends
- Identifies missing values and outliers
- Detects errors or inconsistencies
- Assists in making better decisions for further analysis

3. What are the main differences between Series, DataFrame, and Panel?**

Feature	Series	DataFrame	Panel (Deprecated)
Dimension	1D	2D	3D
Structure	Single column	Table (rows & columns)	Collection of DataFrames
Example	[1, 2, 3]	Table of data	Multiple tables

4. Explain the use of:**

a. isnull()

- Returns True for missing values
- Used to detect missing data

b. notnull()

- Returns True for non-missing values

- Opposite of isnull()

c. dropna()

- Removes rows or columns with missing values

d. fillna()

- Replaces missing values with a specified value (e.g., 0, mean)

5. What is column transformation and why is it important in data analysis?**

Column transformation means modifying or converting column values to make them suitable for analysis.

Examples:

- Changing data type (int to float)
- Applying functions to columns
- Creating new columns

Importance:

- Improves data quality
- Helps in better analysis and modeling
- Makes data consistent and meaningful

```
import pandas as pd
import numpy as np

# 1. Select a single column from a DataFrame
print("Q1: Select a single column")
df1 = pd.DataFrame({"Name": ["Aman", "Riya"], "Marks": [85, 90]})
print(df1["Name"])
print()

# 2. Add a new column with constant values
print("Q2: Add new column with constant value")
df2 = pd.DataFrame({"A": [1,2,3]})
df2["B"] = 100
print(df2)
print()

# 3. Sort a DataFrame by one column
print("Q3: Sort DataFrame by column")
df3 = pd.DataFrame({"Name": ["Aman", "Riya"], "Marks": [90, 80]})
print(df3.sort_values("Marks"))
print()

# 4. Rename DataFrame columns
print("Q4: Rename columns")
```

```

df4 = pd.DataFrame({"A": [1], "B": [2]})
df4.rename(columns={"A": "Col1", "B": "Col2"}, inplace=True)
print(df4)
print()

# 5. Display summary statistics
print("Q5: Summary statistics")
df5 = pd.DataFrame({"Marks": [10, 20, 30, 40]})
print(df5.describe())
print()

# 6. Filter rows using multiple conditions
print("Q6: Filter rows with multiple conditions")
df6 = pd.DataFrame({"Marks": [40, 60, 80], "Age": [18, 20, 22]})
print(df6[(df6["Marks"] > 50) & (df6["Age"] > 18)])
print()

# 7. Group data and calculate mean
print("Q7: Group by and mean")
df7 = pd.DataFrame({"Dept": ["IT", "IT", "HR"], "Salary": [50000, 60000, 40000]})
print(df7.groupby("Dept")["Salary"].mean())
print()

# 8. Merge two DataFrames
print("Q8: Merge DataFrames")
df8a = pd.DataFrame({"ID": [1,2], "Name": ["Aman", "Riya"]})
df8b = pd.DataFrame({"ID": [1,2], "Marks": [85, 90]})
print(pd.merge(df8a, df8b, on="ID"))
print()

# 9. Drop rows and columns
print("Q9: Drop rows and columns")
df9 = pd.DataFrame({"A": [1,2,3], "B": [4,5,6]})
df9 = df9.drop(0) # drop row
df9 = df9.drop("B", axis=1) # drop column
print(df9)
print()

# 10. Apply function to a column
print("Q10: Apply function")
df10 = pd.DataFrame({"Marks": [10, 20, 30]})
df10["Marks"] = df10["Marks"].apply(lambda x: x*2)
print(df10)
print()

# 11. Pivot and melt operations
print("Q11: Pivot and Melt")
df11 = pd.DataFrame({
    "Name": ["Aman", "Riya"],
    "Math": [80, 90],
    "Science": [70, 85]
})

```

```

}))
melted = pd.melt(df11, id_vars=["Name"])
print(melted)
pivoted = melted.pivot(index="Name", columns="variable", values="value")
print(pivoted)
print()

# 12. Multilevel indexing
print("Q12: MultiIndex")
arrays = [["A", "A", "B"], [1, 2, 1]]
index = pd.MultiIndex.from_arrays(arrays)
df12 = pd.DataFrame({"Data": [10, 20, 30]}, index=index)
print(df12)
print()

# 13. Compare merge vs join
print("Q13: Merge vs Join")
df13a = pd.DataFrame({"A": [1, 2]}, index=[0, 1])
df13b = pd.DataFrame({"B": [3, 4]}, index=[0, 1])
print("Join:\n", df13a.join(df13b))
print("Merge:\n", pd.merge(df13a, df13b, left_index=True, right_index=True))
print()

# 14. Custom aggregation function
print("Q14: Custom aggregation")
df14 = pd.DataFrame({"Dept": ["IT", "IT", "HR"], "Salary": [50000, 60000, 40000]})
def my_func(x):
    return max(x) - min(x)

print(df14.groupby("Dept")["Salary"].agg(my_func))
print()

# 15. Time series data wrangling
print("Q15: Time series operations")
dates = pd.date_range("2023-01-01", periods=5)
df15 = pd.DataFrame({"Date": dates, "Value": [10, 20, 30, 40, 50]})
df15.set_index("Date", inplace=True)
print(df15.resample("2D").mean())

```

Q10: Apply function

Marks

0	20
1	40
2	60

Q11: Pivot and Melt

	Name	variable	value
0	Aman	Math	80
1	Riya	Math	90
2	Aman	Science	70
3	Riya	Science	85

	variable	Math	Science
Name			
Aman		80	70
Riya		90	85

Q12: MultiIndex

	Data
A 1	10
2	20
B 1	30

Q13: Merge vs Join

Join:

	A	B
0	1	3
1	2	4

Merge:

	A	B
0	1	3
1	2	4

Q14: Custom aggregation

Dept

HR 0

IT 10000

Name: Salary, dtype: int64

Q15: Time series operations

	Value
Date	
2023-01-01	15.0
2023-01-03	35.0
2023-01-05	50.0

Conclusion –

In this experiment, we learned how to perform data wrangling using Pandas. We practiced selecting, filtering, sorting, and transforming data. We also explored advanced operations like grouping, merging, pivoting, and time series analysis. These techniques help in organizing and transforming raw data into a structured format suitable for analysis

